

Wstęp teoretyczny i założenia do projektu

1. Charakterystyka problemu

Hasła w rzeczywistych systemach nie są przechowywane w postaci jawnej, lecz jako wynik działania funkcji skrótu. System nie zna bezpośrednio tekstu hasła, a jedynie jego hash. Projektowany program nie będzie odwracał funkcji haszującej, lecz będzie generował kolejne hasła, obliczał ich skróty i porównywał je z hashem docelowym. Problem można skutecznie rozproszyć na wiele maszyn. Klienci mogą niezależnie sprawdzać możliwe hasła, a serwer jedynie zarządza przydziałem pracy i zbiera wyniki.

2. Podstawy teoretyczne

2.1. Funkcje skrótu

Funkcja skrótu przekształca dane wejściowe dowolnej długości do ciągu o ustalonej długości. W projekcie można wykorzystać przykładowo SHA-256, ponieważ jest łatwa do użycia w testach laboratoryjnych.

2.2. Atak słownikowy

Metoda słownikowa polega na sprawdzaniu haseł znajdujących się w przygotowanym zbiorze często spotykanych słów i ich wariantów. Do słownika można dodawać modyfikacje, na przykład cyfry na końcu, wielkie litery lub znaki specjalne. Zaletą tej metody jest szybkie uzyskiwanie wyników dla słabych haseł. W systemie rozproszonym słownik można dzielić na fragmenty i przydzielać różnym klientom.

2.3. Atak brute force

Metoda brute force polega na generowaniu wszystkich możliwych kombinacji znaków z ustalonego alfabetu, na przykład: małe litery, wielkie litery, cyfry oraz wybrane znaki specjalne. Jest to metoda pewna, ale kosztowna obliczeniowo. Aby podzielić pracę między klientów, wszystkie możliwe hasła można ponumerować. Serwer przydziela wtedy każdemu klientowi zakres numerów do sprawdzenia. Dzięki temu każdy komputer sprawdza inną część możliwych haseł i praca jest równo podzielona.

2.4. Algorytmy planowane do wykorzystania

Jednym z nich jest funkcja skrótu SHA-256, która będzie używana do obliczania hashy generowanych kandydatów na hasło. Obliczony hash będzie następnie porównywany z hashem docelowym, aby sprawdzić, czy znaleziono właściwe hasło.

Podstawową metodą wyszukiwania będzie brute force, czyli generowanie wszystkich możliwych kombinacji znaków z ustalonego alfabetu i sprawdzanie. Metoda ta pozwala sprawdzić wszystkie możliwe hasła.

Drugą metodą będzie atak słownikowy, polegający na sprawdzaniu haseł znajdujących się w przygotowanym pliku ze słownikiem. Metoda ta umożliwi szybkie znalezienie prostych i często używanych haseł.

Do rozdzielania pracy między komputery zostanie zastosowany podział zakresowy. Serwer będzie przydzielał klientom zakresy kombinacji do sprawdzenia, dzięki czemu praca zostanie rozdzielona pomiędzy wiele maszyn.

Dodatkowo planowane jest wykorzystanie wielowątkowości po stronie klienta, co pozwoli na równoległe wykonywanie obliczeń i lepsze wykorzystanie mocy obliczeniowej procesora.

3. Dotychczasowe podejścia i inspiracje

W praktyce łamanie haseł bywa realizowane z użyciem narzędzi takich jak Hashcat czy John the Ripper. Celem projektu nie jest odtworzenie pełnej funkcjonalności profesjonalnych narzędzi, lecz zbudowanie uproszczonego systemu.

Z punktu widzenia przedmiotu najistotniejsze jest pokazanie mechanizmów systemu rozproszonego: komunikacji klient-serwer, synchronizacji pracy, przydzielania zadań, logowania wyników oraz pomiaru czasu wykonania.

4. Proponowana architektura rozwiązania

Najbardziej praktycznym wariantem jest architektura klient-serwer. Jeden komputer pełni rolę serwera, a pozostałe komputery klienta. Serwer utrzymuje listę połączonych hostów, rozdziela zadania i odbiera wyniki częściowe.

Klienci są odpowiedzialni wyłącznie za wykonywanie obliczeń. Po uruchomieniu łączą się z adresem IP i portem serwera, zgłaszają gotowość do pracy, pobierają paczkę danych, wykonują obliczenia i zwracają informację o zakończeniu zakresu lub znalezieniu poprawnego hasła.

Pliki konfiguracyjne będą wykorzystywane do przechowywania parametrów działania systemu. W pliku znajdują się: adres IP serwera, numer portu, alfabet znaków, minimalna i maksymalna długość hasła, tryb pracy (brute force lub słownikowy) oraz hash hasła, które ma zostać odnalezione.

4.1. Komunikacja i komunikaty

W systemie przewidziane są następujące komunikaty:

TASK - wysyłany przez serwer do klienta. Zawiera informacje o trybie pracy (np. brute force lub słownikowy), początkowym indeksie, liczbie kombinacji do sprawdzenia oraz parametrach zadania. Komunikat ten przydziela klientowi fragment pracy.

RESULT - wysyłany przez klienta po zakończeniu zadania. Zawiera status wykonania, czas obliczeń oraz ewentualnie znalezione hasło.

STOP - wysyłany przez serwer w celu poinformowania klientów o zakończeniu obliczeń, na przykład po odnalezieniu hasła.

5. Pomiary wydajności

Projekt powinien umożliwiać sprawdzenie, jak liczba komputerów i wielkość paczek zadań wpływają na czas obliczeń. Dlatego należy zapisywać: całkowity czas zadania, czas obliczeń po stronie klienta, liczbę sprawdzonych kandydatów oraz nazwę komputera wykonującego dane zadanie. Ponieważ zegary na różnych komputerach mogą się różnić, najbezpieczniej mierzyć czas całkowity po stronie serwera oraz czas obliczeń lokalnych na kliencie.

6. Zakres funkcjonalny projektu

- Aplikacja serwera uruchamiana z konsoli, z możliwością ustawienia portu i parametrów zadania.
- Aplikacja klienta uruchamiana z konsoli, z możliwością podania IP serwera i portu.
- Obsługa trybu słownikowego i brute force.
- Dynamiczny przydział paczek zadań klientom, którzy zgłoszą gotowość.
- Logowanie wyników do pliku: host, czas, zakres, status, ewentualnie znalezione hasło.
- Możliwość uruchamiania na różnej liczbie komputerów.

7. Przypadki użycia

- Użytkownik uruchamia serwer i ustawia parametry zadania.
- Użytkownik uruchamia klientów na kolejnych komputerach i wskazuje adres IP serwera.
- Klient dołącza do systemu i pobiera paczkę obliczeń.
- Klient kończy zakres i pobiera następny fragment pracy.
- Klient znajduje poprawne hasło i raportuje wynik do serwera.
- Serwer zatrzymuje pozostałych klientów i zapisuje końcowy raport z testu.